



VANLANG UNIVERSITY
School of Technology
Information Technology



HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

(Database Management System)

CHƯƠNG 11: KỸ THUẬT TỐI ƯU HÓA TRUY VẤN



GVGD: THS. NGUYỄN THỊ QUYÊN



HỌC KỲ II - NĂM HỌC 2024-2025



KHÓA 29T-IT





NỘI DUNG

01. ĐẶT VẤN ĐỀ

02. NGUYÊN NHÂN TRUY VẤN CHẬM

03. CÁC PHƯƠNG PHÁP CẢI TIẾN

04. KẾT LUẬN



1. ĐẶT VẤN ĐỀ

- Khi những phần mềm có khối lượng dữ liệu lớn như Ngân hàng, Đào tạo, Thương mại điện tử... thì những câu truy vấn sẽ bị ảnh hưởng bởi các yếu tố:
 - Việc kết nhiều bảng với nhiều bản ghi là việc thường xuyên diễn ra
 - Tốc độ thực hiện phần mềm bị chậm.

=> Một trong những phương pháp cần thiết để tốc độ của phần mềm được đảm bảo là phải tối ưu hóa từng dòng lệnh SQL.



2. NGUYÊN NHÂN TRUY VẤN CHẬM

- **Không/thiếu sử dụng các lợi ích của Indexes.**
- **Kết nối mạng chậm.**
- **Các câu truy vấn Transact-SQL chuyển số lượng dữ liệu lớn từ server đến client.**
- **Locks or deadlocks bị cấm.**
- **Trả về các dữ liệu không cần thiết.**
- **Thiếu bộ nhớ**

3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.1 THU HẸP GIÁ TRỊ TRẢ VỀ

- Đây là câu lệnh sẽ khiến SQL quét toàn bộ table, trả về dữ liệu trùng lặp làm tiêu tốn bộ nhớ

```
SELECT * FROM table_1 LEFTJOIN table_2 WHERE table_1.id = table_2.id
```

- Nên cần thu hẹp giới hạn của giá trị trả về sẽ tiết kiệm bộ nhớ khi truyền từ server về client.

```
SELECT table_1.id, table_2.username, table_2.lucky  
FROM table_1 LEFTJOIN table_2 WHERE table_1.id = table_2.id
```



3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.2 KHÔNG HẠN CHẾ INDEX (1/4)

a) Index với toán tử phủ định:

- Index không thể thực hiện với toán tử phủ định do đó các toán tử phủ định sẽ làm chậm câu truy vấn, vì vậy hạn chế sử dụng các toán tử sau:

IS NULL, !=, <>, NOT, NOT EXISTS, NOT IN, NOT LIKE, NOT BETWEEN ... AND

3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.2 KHÔNG HẠN CHẾ INDEX (2/4)

b) Index với toán tử so sánh 2 lần:

- Không nên dùng các toán tử “<=”, “>=” để hạn chế việc so sánh 2 lần, làm chậm câu truy vấn:

```
SELECT userid, username FROM user WHERE user_amount <=3000
```

- Hãy thay thế câu trên như sau:

```
SELECT userid, username FROM user WHERE user_amount < 3001
```

3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.2 KHÔNG HẠN CHẾ INDEX (3/4)

c) Sử dụng toán tử LIKE hợp lý:

- Nếu toán tử so sánh với LIKE dư thừa, không hợp lý sẽ làm cho Index không thực hiện được:

```
SELECT lname, fname, address FROM Customers WHERE fname LIKE '%V%'
```

- Hãy thay thế câu trên như sau:

```
SELECT lname, fname, address FROM Customers WHERE fname LIKE 'V%'
```

3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.2 KHÔNG HẠN CHẾ INDEX (4/4)

d) Hạn chế sử dụng hàm trên cột Index:

- Nếu sử dụng hàm DATEDIFF trên cột dateofbirth thì Index gắn trên cột này không chạy được:

```
SELECT member_number, first_name, last_name  
FROM members  
WHERE DATEDIFF(yy, dateofbirth, GETDATE()) > 21
```

- Hãy thay thế câu trên như sau:

```
SELECT member_number, first_name, last_name  
FROM members  
WHERE dateofbirth < DATEADD(yy, -21, GETDATE())
```

3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.3 KHÔNG ĐỂ SQL THỰC HIỆN THAO TÁC THỪA

a) Dùng UNION ALL thay cho UNION:

- Cả UNION và UNION ALL đều quét table như nhau nhưng UNION tiêu tốn phần lớn thời gian cho thao tác “Distinct sort” để loại bỏ những bản ghi trùng nhau (mặc dù không hề có những records như vậy):

```
SELECT userId, userName, userPhone FROM user WHERE dept = 'ruby'
```

UNION

```
SELECT userId, userName, userPhone FROM user WHERE rank = '5'
```

- Hãy thay thế câu trên như sau:

```
SELECT userId, userName, userPhone FROM user WHERE dept = 'ruby'
```

UNION ALL

```
SELECT userId, userName, userPhone FROM user WHERE rank = '5'
```

```
SELECT userId, userName, userPhone FROM user WHERE dept = 'ruby' or rank = '5'
```



3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.3 KHÔNG ĐỂ SQL THỰC HIỆN THAO TÁC THỪA

b) Hạn chế dùng DISTINCT, ORDER BY khi không cần thiết:

- **DISTINCT** sẽ kiểm tra và loại bỏ các bản ghi trùng lặp, **ORDER BY** sẽ sort các records hầu hết các trường hợp, 2 thao tác này chiếm phần lớn thời gian truy vấn nên hạn chế không dùng khi không cần thiết.

3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.3 KHÔNG ĐỂ SQL THỰC HIỆN THAO TÁC THỪA

c) Dùng IF EXISTS thay cho COUNT() hoặc COUNT(DISTINCT):

- Khi xác định sự tồn tại của records trong bảng nên dùng IF EXISTS thay cho COUNT() hoặc COUNT(DISTINCT) vì hàm COUNT sẽ tìm cách lấy tất cả phần tử, so sánh, đếm nên sẽ tốn tài nguyên và nhiều thao tác hơn.

```
IF (SELECT COUNT(*) FROM table_name WHERE column_name = 'abc')
```

- Hãy thay thế câu trên như sau:

```
IF EXISTS (SELECT * FROM table_name WHERE column_name = 'abc')
```



3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.4 SỬ DỤNG STORED PROCEDURE

- **Đối với các thao tác được thực hiện một cách thường xuyên và có xử lý phức tạp nên sử dụng Stored Procedure (SP) để:**
 - Giảm lượng dữ liệu truyền đến Server do SP được lưu sẵn ở phía server do đó không cần phải gửi cả câu lệnh SQL dài tới server mà chỉ cần gửi tham số.
 - SP được biên dịch ngay ở lần đầu chạy, những lần sau chạy SP sẽ sử dụng lại file đã biên dịch trước đó nên tốc độ sẽ nhanh hơn.
 - Có thể dùng vòng for trong SP để gọi nhiều câu lệnh SQL gửi lên server điều này giúp tái sử dụng source.



3. CÁC PHƯƠNG PHÁP CẢI TIẾN

3.5 SỬ DỤNG INDEX HỢP LÝ

- Số Index không nên quá nhiều, khoảng 4-5 index trên một bảng.
- Nên tạo Index trên các cột mà có giá trị là Integer hơn là giá trị chuỗi.
- Nếu tạo ra một index gồm nhiều cột, thứ tự của những cột là khóa rất quan trọng
- Nếu muốn dùng lệnh Join nhiều bảng thì tạo các khóa đại diện có kiểu dữ liệu là Integer và tạo index trên những cột đó.
- Tạo khóa chính đại diện là kiểu Integer.
- Nếu ứng dụng thực hiện cùng một truy vấn nhiều lần trên một bảng, nên xem xét việc tạo một index trên toàn bảng.



4. KẾT LUẬN

- Việc chỉnh sửa lại câu truy vấn không tốn nhiều công sức nhưng sẽ giúp hệ thống giảm thiểu tính toán đáng kể.
- Khi sử dụng Index sẽ giúp truy vấn nhanh hơn nhưng ngược lại làm chậm thao tác INSERT, UPDATE do đó hãy cân nhắc toàn bộ hệ thống trước khi áp dụng.
- Nên thường xuyên tập thói viết tối ưu để biến thành thói quen, giúp chúng ta tận dụng tốt hơn tài nguyên máy chủ trong quá trình xử lý câu lệnh.



VANLANG UNIVERSITY
School of Technology
Information Technology

